SQL Query Execution Time & Cost Predictor

Comprehensive Project Review, Architecture Guide & Viva Master Preparation

Stack: FastAPI + React + PostgreSQL | ML: HistGradientBoosting & Random Forest | R^2 Score: 94.62% | Active Learning: Online History Retraining | Tooling: Docker Compose

1. The 30-Second Elevator Pitch

> **Memorize This for Your Opening Statement:**
>
> *"Our project is an AI-driven SQL Query Performance Predictor and Optimizer. Traditional relational databases like PostgreSQL use static mathematical cost formulas that often fail on complex multi-table joins or skewed data distributions. We built an end-to-end platform that extracts **29 deep structural and relational plan features** from PostgreSQL's JSON query execution trees and feeds them into trained Machine Learning models (Histogram Gradient Boosting and Random Forest) to predict actual execution latency in milliseconds before running the query. Beyond prediction, the platform includes real-time **ReactFlow plan visualization**, **automated SQL rewriting**, and **isolated What-If index simulations** to diagnose and resolve database bottlenecks."*

2. Problem Statement & Motivation

Database Query Optimizers (such as PostgreSQL's Cost-Based Optimizer) choose execution paths based on mathematical cost formulas derived from statistics (pg_statistic). While effective for basic queries, these estimators face fundamental challenges:

CHALLENGE	POSTGRESQL NATIVE COST ESTIMATOR	OUR MACHINE LEARNING APPROACH
Measurement Unit	Arbitrary unitless cost points (e.g. 100.5). Cost does *not* equal milliseconds.	Predicts **wall-clock time directly in milliseconds (ms)**.
Hardware Adaptation	Static disk/CPU page costs (`seq_page_cost = 1.0`). Misses memory bus & CPU cache effects.	Learns empirical execution behavior on the target physical/cloud hardware.
Estimation Drift	Assumes column independence; errors multiply exponentially across multi-table joins.	Ensemble trees capture high-order non-linear interactions across plan nodes.
Proactive Tuning	Static advisory; requires manual DBA intervention to diagnose slow queries.	Real-time bottleneck hotspot detection + automated index simulation with zero risk.

3. End-to-End System Architecture

The system is deployed as three decoupled, containerized services using Docker Compose:

```
+-------------------------------------------------------------------------------+ | REACT 18 FRONTEND (Port 5173) | | Monaco SQL Editor | ReactFlow Plan Tree | Recharts Visuals | What-If Simulation UI| +---------------------------------------+---------------------------------------+ | REST API (JSON) +---------------------------------------v---------------------------------------+ | FASTAPI BACKEND (Port 8000) | | 1. SQL Validator (Blocks DDL/DML) 2. EXPLAIN JSON Parser 3. 29 Feature Engine | | 4. ML Model Inference Engine 5. Rule-Based Rewriter 6. History Collector | +---------------------------------------+---------------------------------------+ | SQLAlchemy Core +---------------------------------------v---------------------------------------+ | POSTGRESQL 15 DATABASE (Port 5432) | | Relational Schema (customers, orders, items) | Transactional Rollback Index Tests | +-----------------------------------------------+
```

- **PostgreSQL 15 Container:** Hosts synthetic e-commerce schema with 250,000+ relational records across `customers`, `orders`, `order_items`, `products`, and `payments`, plus query history tracking.
- **FastAPI Microservice:** Asynchronous Python backend executing plan extraction, AST SQL validation, feature engineering, ML inference, and automated optimizer rules.
- **Vite + React Frontend:** High-performance SPA with Monaco editor, interactive drag-and-drop plan tree visualizer, and live monitoring dashboards.

4. Machine Learning Pipeline & Mathematical Foundations

The model does not treat SQL as plain text (like NLP). Instead, it evaluates the **physical relational execution plan** generated by the query compiler. This guarantees query-invariant representations regardless of SQL syntax formatting.

A. The 29 Extracted Features (Feature Engineering)

Our recursive parser (`feature_extractor.py`) walks the entire PostgreSQL plan tree and extracts 29 numerical and structural features:

`startup_cost` `total_cost` `plan_rows` `plan_width` `parallel_aware` `plan_depth` `number_of_plan_nodes` `number_of_tables` `number_of_joins` `number_of_filters` `number_of_sort_nodes` `number_of_aggregate_nodes` `number_of_seq_scans` `number_of_index_scans` `number_of_nested_loop_joins` `number_of_hash_joins` `number_of_merge_joins` `total_scanned_rows` `max_intermediate_rows` `total_tree_cost` `has_hash_join` `has_bitmap_scan` `number_of_bitmap_scans` `max_plan_width` `has_limit` `number_of_limit_nodes` `has_materialize` `output_to_scanned_ratio` `is_unbounded_scan`

B. Mathematical Formulation & Target Log-Transformation

Query execution latency spans 4 to 5 orders of magnitude (from 0.05 ms to 500+ ms). Directly minimizing Mean Squared Error (MSE) would cause the regressor to fit only massive queries while ignoring smaller queries. To resolve this, we apply a **Log1p transformation**:

```
y_transformed = log(1 + actual_execution_time_ms) prediction = exp(model.predict(features)) - 1
```

C. Model Tournament Benchmark & Production Selection

We trained and evaluated three champion regression algorithms on 2,500 real execution profiles:

MODEL	R^2 SCORE (ACCURACY)	MAE (MEAN ABSOLUTE ERROR)	RMSE	MAPE (NORMALIZED)	TRAINING SPEED	STATUS
Histogram Gradient Boosting	**94.62%**	**3.92 ms**	**10.56 ms**	**23.41%**	**11.27 s**	Champion (Active)
Random Forest Regressor	94.50%	4.01 ms	10.67 ms	26.17%	0.66 s	Benchmarked
XGBoost (Hist)	93.60%	4.14 ms	11.52 ms	36.00%	19.65 s	Benchmarked

Why HistGradientBoosting won: Native histogram binning handles continuous relational plan costs with minimal memory footprint while achieving the highest R^2 (0.9462).

5. Deep Dive: The Database Performance Dashboard (`/`)

The Dashboard serves as the command center for monitoring query traffic, server health, and model performance over time. It is driven by the `/api/dashboard/analytics` endpoint.

TOTAL QUERIES	AVG ACTUAL LATENCY	AVG PREDICTED LATENCY	HIGH COST ALERTS
2,500+	**12.4 ms**	**11.8 ms**	**Low (3%)**
Captured with plan telemetry	Real database execution	ML model inference time	Queries running > 500ms

Dashboard Components Explained:

- **KPI Stat Cards:** Summarize high-level execution volume, mean actual vs. predicted execution latency, and overall normalized error percentage.
- **Query Execution Latency Distribution:** A Recharts bar histogram categorizing queries into execution time buckets (`<1ms`, `1-5ms`, `5-20ms`, `20-100ms`, `>100ms`). This visually proves that the workload reflects real-world multi-tiered database operations.
- **Prediction Accuracy Trend Chart:** A dual-line chart plotting actual vs. predicted execution time across recent queries, highlighting how closely the ML inference tracks actual PostgreSQL execution.
- **Workload Category Breakdown:** Donut/pie distribution showing the proportion of *Fast (<5ms)*, *Moderate (5-50ms)*, and *Expensive (>50ms)* queries.
- **Slowest Queries Diagnostic Table:** Ranks top resource-consuming queries with a one-click "*Analyze Query*" button that loads the problematic query directly into the Analyzer for tuning.

6. Deep Dive: Model Performance & Active Learning (`/metrics`)

The Model Metrics page provides complete transparency into machine learning behavior, comparative model evaluation, and continuous learning capabilities. Driven by `/api/model/metrics` and `/api/model/retrain`.

CHAMPION ALGORITHM	R^2 ACCURACY SCORE	DEEP FEATURE VECTORS
HistGradientBoosting	**94.62%**	**29 Features**
Target: Log-Scale (log1p)	Explains 94.6% of latency variance	Cost, scan, join, and size metrics

Key Capabilities of the Metrics Page:

- **Model Tournament Leaderboard:** Compares MAE, RMSE, R^2 Score, and Training Time across all algorithms, explicitly identifying which model is currently active in production.
- **Error Metric Comparison Bar Chart:** Visualizes MAE and RMSE side-by-side using Recharts, making it easy to explain to reviewers why the champion model was selected.
- **Active Feature Vectors Display:** Lists all 29 structural plan features currently consumed by the model, showing examiners that feature engineering goes far deeper than basic SQL length or table count.
- **Continuous Active Retraining ("Self-Tune from History"):**
  A distinctive capability of this project is its ability to learn continuously. As users execute new queries in the Analyzer, their execution telemetry (plan tree, features, and actual execution time) is saved to the `query_history` table. Clicking **"Self-Tune from History"** triggers an asynchronous retraining pipeline that:
  1. Extracts newly recorded execution samples from the database.
  2. Appends them to the balanced training dataset.
  3. Re-fits the models and compares new evaluation scores.
  4. Hot-swaps the production `best_model.joblib` artifact without server restart.

7. Deep Dive: Query Analyzer, Visualizer & Optimizer (`/analyzer`)

The Query Analyzer is the core interactive interface of the system, combining five specialized capabilities in one unified view:

1. MONACO SQL EDITOR & QUICK EXAMPLES

Industry-standard code editor with syntax highlighting, auto-formatting, and a one-click Quick Examples bar containing 6 diverse queries (Joins, Date Ranges, Point Scans, Unindexed Scans, Lookups, and CTE Stress Tests).

2. REAL-TIME ML PREDICTOR & TELEMETRY

Displays predicted execution time (ms), PostgreSQL native estimated cost, and estimated rows. Clicking "Execute & Validate" triggers live database execution to measure actual duration and validation error.

3. INTERACTIVE REACTFLOW PLAN TREE

Visual graph mapping out the execution tree from table scans to final output. Nodes feature dynamic color-coded hotspot detection:

- 🔴 **Critical Hotspot:** Consumes >40% of plan cost/time.
- 🟡 **Moderate Load:** Consumes 15-40% of cost/time.
- 🟢 **Fast:** Low-cost indexed lookup or node.

4. WHAT-IF INDEX SIMULATION & REWRITING

Identifies missing indexes on sequential scans. Provides a **Simulate Index** button that tests index creation in an isolated database transaction with rollback. Also features rule-based SQL rewriting (e.g. anti-pattern subquery to JOIN).

8. Step-by-Step Live Demonstration Walkthrough

> **Follow this exact sequence during your demonstration to impress the reviewers:**

1. **Step 1: Open the Query Analyzer (`/analyzer`)**
   Show the interface and point to the **Quick Example Queries** bar at the top. Click on **"Customer Orders"** (a 2-table join). Point out the Monaco editor syntax and show how the editor loads cleanly.
2. **Step 2: Run "Execute & Validate"**
   Click the green **"Execute & Validate"** button. Point to the results panel:
   - Show the **PostgreSQL Estimated Cost** (e.g. ~145.2).
   - Show the **ML Predicted Execution Time** (e.g. ~0.19 ms).
   - Show the **Actual Execution Time** (e.g. ~0.18 ms) and highlight the accurate correlation.
3. **Step 3: Showcase the ReactFlow Execution Tree**
   Scroll down to the interactive query plan tree. Zoom in/out and drag the canvas. Show how data flows from the sequential/index scan nodes at the bottom up through the Hash Join into the final output node.
4. **Step 4: Demonstrate Bottleneck Detection & What-If Simulation**
   Click the **"Unindexed Status Scan"** example. The query filters on `status = 'CANCELLED'` on the `orders` table. Run it.
   - Show how the plan visualizer highlights the Sequential Scan node as a **Critical Hotspot (in Red)**.
   - Open the **Optimizer & Index Advisor** panel below. Show the recommendation: "*Create index on orders(status)*".
   - Click **"Simulate Index"**. Explain to the professor: "*The system just executed a What-If simulation inside an isolated transaction and rolled it back — showing the exact cost reduction without altering our production schema.*"
5. **Step 5: Navigate to the Dashboard (`/`) and Model Metrics (`/metrics`)**
   Switch to the Dashboard tab to show historical query volume and execution distributions. Then switch to the Metrics tab and highlight the **94.62% R^2 Score** across 2,500 samples, explaining that the project benchmarks multiple algorithms.

9. Top 10 Viva Questions & Winning Answers

Q1: Why do we need Machine Learning when PostgreSQL already has a built-in cost estimator?

Winning Answer: PostgreSQL's native cost estimator uses static, theoretical formulas based on arbitrary cost units (e.g. 1.0 for sequential page cost). It does not output milliseconds. A cost of 100 could take 0.5ms on an NVMe SSD or 40ms under heavy disk I/O. Furthermore, static estimators assume column independence, leading to massive estimation drift on complex multi-table joins. Our ML model learns from empirical execution runs on actual hardware and directly predicts wall-clock execution time in milliseconds.

Q2: Why did you choose Histogram Gradient Boosting and Random Forest over Deep Neural Networks?

Winning Answer: Research consistently shows that for tabular, structured numerical data with non-linear relationships, tree-based ensemble models (Gradient Boosting and Random Forest) outperform Deep Neural Networks. They achieve higher accuracy with lower sample requirements, train in seconds instead of hours, do not require GPU hardware, and provide direct feature interpretability without the risk of overfitting.

Q3: Why did you apply a Log1p transformation to the target variable?

Winning Answer: SQL query execution times exhibit a heavy-tailed, exponential distribution spanning from 0.05 ms for indexed lookups to over 500 ms for large scans. If we trained directly on raw milliseconds, the regression loss function (MSE) would be dominated by high-latency queries, completely ignoring accuracy on sub-millisecond queries. By taking $y = \log(1 + \text{time})$, we normalize the variance and ensure balanced sensitivity across both fast and slow queries.

Q4: Why can percentage error look high on sub-millisecond queries?

Winning Answer: For queries executing in 0.08 ms, a difference of just 0.05 ms produces a mathematical error of over 60% due to the near-zero denominator. In real-world database systems, a 0.05 ms variance is completely negligible microsecond noise. That is why statistical standards for query regression focus on **Mean Absolute Error (MAE = 3.92 ms)** and **R^2 Score (0.9462)** rather than raw percentage error.

Q5: How does database buffer caching affect the ML prediction?

Winning Answer: PostgreSQL caches disk pages in its Shared Buffers RAM. The first time a query executes (cold cache), it may experience disk I/O latency; subsequent executions (warm cache) read directly from memory and run much faster. In our backend, we prime the buffer cache to measure consistent steady-state execution time, eliminating random disk latency spikes.

Q6: How does the What-If Index Simulation work without affecting production data?

Winning Answer: We leverage PostgreSQL's transactional DDL support. We open an explicit transaction block (`BEGIN`), create the suggested index, run `EXPLAIN` to measure the cost reduction on the new index, and immediately issue a `ROLLBACK`. The index is never permanently committed, zero production storage is used, and no locks are held.

Q7: How do you prevent SQL Injection and destructive queries?

Winning Answer: We implemented a strict whitelist validator in `sql_validator.py`. The backend verifies that incoming queries strictly begin with `SELECT` or `WITH` statements. Any query containing destructive keywords such as `DROP`, `DELETE`,

`UPDATE`, `INSERT`, `ALTER`, or `TRUNCATE` is immediately rejected before reaching PostgreSQL.

Q8: How does the system handle continuous learning and model drift?

Winning Answer: The system logs query execution telemetry into `query_history`. Through the "*Self-Tune from History*" feature on the Metrics page, the backend extracts recent execution profiles, appends them to the balanced training dataset, and re-fits the model to adapt to newly introduced query patterns or data volume changes.